

SMC PLIC — CCI Integration Guide

SystemC CCI — What It Is, Why It Matters, and How to Use It in the PLIC Model

Document: 04_CCI_Integration_Guide.md

Project: SMC PLIC (SystemC/TLM-2.0 Model)

Standard Referenced: SystemC CCI 1.0 LRM (Accellera, June 2018)

CCI Installation: `/Users/pdroy/cci`

Table of Contents

1. What is CCI?
 2. Core Concepts
 3. CCI Use Cases (General)
 4. How CCI Applies to the PLIC Model
 5. Concrete API Examples for PLIC
 6. Benefits of Adopting CCI in This Project
 7. Migration Path: `plic_cfg` → CCI Parameters
 8. Build Integration
 9. Summary
-

1. What is CCI?

SystemC Configuration, Control and Inspection (CCI) is an Accellera-standardised C++ API layer (version 1.0, June 2018) built on top of IEEE Std 1666-2011 (SystemC). Its formal purpose is:

“To provide a standard for developing configurable SystemC models and supporting the development of configuration tools.” — CCI 1.0 LRM §1.2

In plain terms, CCI solves a pervasive problem in SystemC-based Virtual Platforms (VPs) and TLM models: **every IP block designer invented their own ad-hoc mechanism for exposing, setting, and introspecting model parameters**. The result was a proliferation of custom configuration structs, environment variables, command-line parsers, and constructor arguments — none of them interoperable.

CCI replaces this fragmentation with a **single, standardised, broker-mediated parameter infrastructure** that works the same way across models from different vendors and projects.

What CCI is NOT

- It is not a file format (no YAML/JSON schema is mandated; JSON serialisation is supported via `cci_value::to_json()` / `from_json()` but format choice is left to tools).
- It is not a run-time configuration database replacement (it is a C++ library integrated at elaboration and simulation time).
- It is not specific to any EDA tool — it is an open standard with a reference implementation (used in this project under `/Users/pdroy/cci`).

2. Core Concepts

Understanding CCI requires familiarity with five key abstractions:

2.1 Parameter (`cci_param<T>`)

A **parameter** is a named, typed, configurable value owned by a `sc_module`. It is the fundamental unit of configuration.

```
// Declare a mutable integer parameter with a default value and description
cci_param<unsigned> num_sources_p{
    "num_sources",          // name (relative to enclosing module)
    336,                    // default value
    "Number of interrupt sources (1..1023)"
};
```

Parameters are: - **Typed at compile time** (`cci_param<T>`) but also accessible via a type-erased variant (`cci_value`). - **Registered automatically** with the nearest broker upon construction. - Usable like normal variables (`p = p + 1 ; if (p > 8) { ... }`). - Optionally **immutable** (`CCI_IMMUTABLE_PARAM`) or **locked** at run-time.

2.2 Broker (`cci_broker_if` / `cci_broker_handle`)

A **broker** is a registry that aggregates parameters and manages preset values. There is always one **global broker**; modules may optionally register **local brokers** to keep internal parameters private.

```
Global broker
├─ top.plic.num_sources      ← registered here if no local broker
├─ top.plic.num_contexts
└─ top.cpu.freq_mhz
```

The broker is the point of contact for tools: a configuration file loader, a testbench stimulus generator, or a regression script all talk to the broker to inject preset values before modules elaborate.

2.3 Preset Value

A **preset value** is a value injected into the broker *before* the owning module is constructed. When the parameter's constructor runs, it automatically picks up the preset instead of its default.

```
// In sc_main or a platform integrator, BEFORE constructing plic:
broker.set_preset_cci_value("top.plic.num_sources", cci_value(128));

// Now constructing plic: num_sources_p == 128, not 336
smc::plic dut("plic");
```

This is the primary mechanism for **tool-driven and testbench-driven configuration**.

2.4 Handle (`cci_param_typed_handle<T>` / `cci_param_untyped_handle`)

External code (testbenches, monitors, other modules) never touches a parameter directly. They obtain a **handle** from the broker via name lookup:

```
auto h = broker.get_param_handle<unsigned>("top.plic.num_sources");
if (h.is_valid()) {
    unsigned n = h.get_value();    // typed read
}
```

Handles track originator identity, are automatically invalidated when the parameter is destroyed, and carry the same read/write API as the parameter itself.

2.5 Originator (`cci_originator`)

Every read and write through CCI carries an **originator** — the identity (sc_object pointer or string name) of whoever is making the access. This gives complete traceability:

“Parameter `plic.num_sources` was changed from 336 to 128 by `tb.cfg_loader`.”

3. CCI Use Cases (General)

The CCI LRM identifies five stakeholder perspectives. The table below maps them to concrete activities:

Perspective	What CCI Enables
Tools	EDA platforms enumerate all parameters in a simulation, render a configuration GUI, inject values from a YAML/JSON file, log changes.
IP Provider (model owner)	Declare parameters in the module; they are automatically visible to any compliant tool or testbench without custom glue code.
Testbench / verification	Programmatically set any parameter by name, register callbacks triggered on read/write, enumerate and dump all current values.
System integrator	Use local brokers to selectively expose/hide sub-module parameters; set per-instance preset values at the platform level.
Infrastructure	Adapt legacy parameter solutions (custom structs, environment variables) to the standard interface once, reuse everywhere.

3.1 Use Case: Tool-Driven Pre-Simulation Configuration

A configuration file is parsed and preset values are pushed into the global broker before `sc_start()`. Every module that uses CCI parameters picks them up automatically during elaboration — no module modification needed.

3.2 Use Case: Regression / Sweep Testing

A test harness iterates over a matrix of parameter values. For each configuration it: 1. Creates a fresh global broker. 2. Sets a batch of preset values. 3. Constructs the DUT. 4. Runs simulation. 5. Queries parameter values post-simulation for coverage logging.

3.3 Use Case: Run-Time Introspection and Debugging

A monitor module registers a `post_write_callback` on any parameter. Whenever firmware writes a new threshold or enables new interrupt sources, the callback fires — enabling cycle-accurate logging without modifying the DUT model.

3.4 Use Case: Parameter Locking for Safety

A security-sensitive parameter can be locked after elaboration so that no further modifications are accepted:

```
num_sources_p.lock(); // no password → permanently immutable for this run
```

3.5 Use Case: IP Packaging with Privacy

A sub-IP may expose only selected parameters to the integrator by registering a local broker:

```
cci_utils::broker localBroker("plicBroker");  
cci_register_broker(localBroker);  
localBroker.expose.insert("plic.num_sources"); // visible outside  
// plic.num_contexts is private and invisible to the global broker
```

4. How CCI Applies to the PLIC Model

4.1 Current State of Configuration in the PLIC

The PLIC model currently uses a plain C++ configuration struct passed at construction time:

```
// include/plic.h
struct plic_cfg {
    unsigned num_sources = 336;    // 1..1023
    unsigned num_contexts = 8;     // ≥ 1
    // static constexpr address layout omitted
};

explicit plic(sc_core::sc_module_name name, plic_cfg cfg = plic_cfg());
```

The testbench instantiates it as:

```
smc::plic_cfg cfg{};    // defaults only
smc::plic dut("plic", cfg);
```

Limitations of the current approach:

Limitation	Impact
Configuration is opaque to tools	No EDA tool can enumerate or override PLIC parameters without reading C++ source.
No run-time introspection	A testbench cannot query <code>num_sources</code> by name at run-time.
No change traceability	Nothing records who set which value and when.
No preset mechanism	To change defaults the caller must construct a custom <code>plic_cfg</code> — no late-binding from a config file.
No callback support	A monitor cannot react to parameter changes without modifying the DUT.
<code>access_delay_</code> is private	The 2 ns access latency is hardcoded and not reachable from the outside without source modification.
No parameter descriptions	Tool introspection has no human-readable documentation for any PLIC knob.

4.2 PLIC Parameters Well-Suited for CCI

The following PLIC configuration values are natural CCI parameter candidates:

CCI Parameter Name	Type	Default	Description
<code>plic.num_sources</code>	unsigned	336	Number of interrupt sources (1..1023). Must match <code>PRIORITY[337]</code> in <code>plic.rdl</code> .
<code>plic.num_contexts</code>	unsigned	8	Number of interrupt contexts (harts × privilege modes). SMC: 4 cores × {M, S} = 8.
<code>plic.access_delay_ns</code>	double	2.0	TLM register access latency in nanoseconds.
<code>plic.priority_width</code>	unsigned	3	Width of PRIORITY and THRESHOLD fields in bits (currently 3 per <code>plic.rdl</code>).

4.3 PLIC Testbench Scenarios Enabled by CCI

Once parameters are CCI-managed, the testbench gains powerful new capabilities:

a) Name-based parameter override without recompilation:

```
// Configured from a JSON file or command-line before dut is constructed
broker.set_preset_cci_value("plic.num_sources", cci_value(64u));
broker.set_preset_cci_value("plic.num_contexts", cci_value(4u));
smc::plic dut("plic");
// dut is built with 64 sources and 4 contexts
```

b) Post-simulation parameter dump for traceability:

```
for (auto& h : broker.get_param_handles()) {
    std::cout << h.name() << " = " << h.get_cci_value()
               << " (origin: " << h.get_value_origin().name() << ")\n";
}
```

c) Callback-driven verification of `access_delay_ns`:

```
auto h = broker.get_param_handle<double>("plic.access_delay_ns");
h.register_post_write_callback([](const cci_param_write_event<double>& ev) {
    SC_REPORT_INFO("plic_tb", ("access_delay changed: "
        + std::to_string(ev.new_value) + " ns").c_str());
});
```

d) Regression sweep over source counts:

```
for (unsigned n : {32u, 64u, 128u, 336u}) {
    cci::cci_register_broker(new cci_utils::consuming_broker("Global"));
    broker.set_preset_cci_value("plic.num_sources", cci_value(n));
    run_plic_test();
}
```

5. Concrete API Examples for PLIC

5.1 Declaring CCI Parameters Inside `plic`


```

// plic.h (modified)
#include <cci_configuration>

namespace smc {

SC_MODULE(plic) {
public:
    // ... ports unchanged ...

    SC_CTOR(plic)
    : num_sources_p("num_sources", 336,
                    "Number of interrupt sources (1..1023). "
                    "Source IDs are 1-based; source 0 is reserved.")
    , num_contexts_p("num_contexts", 8,
                    "Number of interrupt contexts (harts × privilege
                    modes). "
                    "SMC default: 4 cores × {M-mode, S-mode} = 8.")
    , access_delay_p("access_delay_ns", 2.0,
                    "TLM register access latency in nanoseconds.")
    {
        // Validate after preset resolution
        SC_METHOD(validate_cfg);
        sensitive << sc_core::SC_ZERO_TIME; // runs once at start of
        simulation
    }

private:
    cci::cci_param<unsigned> num_sources_p;
    cci::cci_param<unsigned> num_contexts_p;
    cci::cci_param<double> access_delay_p;

    void validate_cfg() {
        SC_REPORT_INFO("plic", ("num_sources=" +
                                std::to_string(static_cast<unsigned>(num_sources_p)).c_str()));
        if (num_sources_p < 1 || num_sources_p > 1023)
            SC_REPORT_FATAL("plic", "num_sources out of range [1..1023]");
        if (num_contexts_p < 1)
            SC_REPORT_FATAL("plic", "num_contexts must be >= 1");
    }
};

} // namespace smc

```

5.2 Platform / Testbench Setup

```
// plic_tb.cpp (modified sc_main)
#include <cci_configuration>
#include <cci/utils/consuming_broker.h>

int sc_main(int argc, char* argv[]) {
    // 1. Register global broker (required before any cci_param is
    //    constructed)
    cci::cci_register_broker(new cci_utils::consuming_broker("GlobalBroker"));

    // 2. Inject preset values (optional - override defaults before DUT
    //    exists)
    cci::cci_originator cfg_tool("config_loader");
    auto broker = cci::cci_get_global_broker(cfg_tool);
    broker.set_preset_cci_value("plic.num_sources", cci::cci_value(128u));
    broker.set_preset_cci_value("plic.access_delay_ns", cci::cci_value(4.0));

    // 3. Instantiate DUT - picks up presets automatically
    smc::plic dut("plic");

    // 4. Verify via handle
    auto h_src = broker.get_param_handle<unsigned>("plic.num_sources");
    assert(h_src.is_valid() && h_src.get_value() == 128u);

    sc_core::sc_start();
    return 0;
}
```

5.3 Read Callback for Interrupt Threshold Monitoring

```
// Register a post-write callback on num_contexts (example: enforce even
// count)
auto h = cci::cci_get_broker().get_param_handle<unsigned>
("plic.num_contexts");
h.register_pre_write_callback(
    [] (const cci::cci_param_write_event<unsigned>& ev) {
        if (ev.new_value % 2 != 0) {
            SC_REPORT_WARNING("plic_tb",
                "num_contexts should be even (harts × 2 privilege modes)");
        }
    }
);
```

5.4 Enumerate All PLIC Parameters (Introspection)

```

auto broker = cci::cci_get_broker();
for (auto& h : broker.get_param_handles()) {
    std::string name = h.name();
    // Filter to plic parameters only
    if (name.find("plic.") == 0) {
        std::cout << std::left << std::setw(40) << name
                  << " = " << h.get_cci_value()
                  << " [" << h.get_description() << "]\n";
    }
}

```

6. Benefits of Adopting CCI in This Project

6.1 Interoperability with EDA Tools

Any compliant SystemC simulator or platform tool (e.g., Synopsys Virtualizer, Cadence Xcelium VP) can enumerate, display, and override PLIC parameters from a configuration GUI or scripting interface — without touching PLIC source code. The model becomes a true plug-and-play IP component.

6.2 Separation of Configuration from Implementation

Today, changing `num_sources` from 336 to 128 requires modifying `plic_cfg` and recompiling. With CCI, the value is injected at platform-level or via a config file, and the PLIC itself stays unchanged. This separates **model implementation** from **platform-specific configuration**, reducing maintenance burden.

6.3 Testbench Flexibility Without Recompilation

CCI enables parameter sweeps, corner-case tests, and regressions driven entirely from test scripts or JSON configuration files. Different test scenarios can exercise different PLIC topologies without recompiling the model:

```

# Hypothetical config file (CCI tool would parse and call
set_preset_cci_value)
plic.num_sources    = 64
plic.num_contexts   = 4
plic.access_delay_ns = 10.0

```

6.4 Complete Parameter Provenance and Audit Trail

Every parameter write records *who* changed it and *from what value*. This is invaluable during debug:

- “Was `num_sources` set to 64 by the testbench or by a firmware write?”
- “Which configuration loaded `access_delay_ns = 10`?”

`h.get_value_origin().name()` answers these questions directly.

6.5 Validation and Safety via Callbacks

Pre-write callbacks can enforce invariants centrally — e.g., reject `num_sources > 1023` before the value is committed — rather than scattering validation across constructors and transport functions.

6.6 Documentation Embedded in the Model

Parameter descriptions are first-class citizens of the CCI API. A tool calling `h.get_description()` retrieves “Number of interrupt sources (1..1023). Source IDs are 1-based; source 0 is reserved.” — removing the need to chase down comments in headers.

6.7 Metadata for Advanced Tooling

CCI parameters support attaching arbitrary key-value metadata:

```
num_sources_p.add_metadata("rdl_field", cci::cci_value("PRIORITY[337]"));
num_sources_p.add_metadata("fw_define",
    cci::cci_value("PLIC0_MAX_INTERRUPTS"));
num_sources_p.add_metadata("range_min", cci::cci_value(1u));
num_sources_p.add_metadata("range_max", cci::cci_value(1023u));
```

This metadata is queryable at run-time, enabling documentation generators, hardware abstraction layers, and coverage tools to consume it automatically.

6.8 Zero Overhead on Hot Paths

CCI parameter access uses C++ typed references under the hood. The `operator=` and arithmetic operators on `cci_param<T>` resolve to the underlying `T` directly. For simulation-critical paths (e.g., the interrupt arbitration loop), using `unsigned n_src = num_sources_p;` is as fast as a plain variable read after the first elaboration.

6.9 Standards Compliance Signals IP Maturity

Adopting CCI signals to downstream integrators and EDA partners that the PLIC model follows industry standards. This reduces integration friction and audit risk when the model is shared across teams or shipped to customers.

7. Migration Path: `plic_cfg` → CCI Parameters

The migration is incremental and non-breaking. The existing `plic_cfg` struct can remain for backward compatibility while CCI parameters are introduced alongside it.

Phase 1 — Add CCI Parameters (non-breaking)

Keep `plic_cfg` constructor. Add `cci_param` members that are initialised from the struct:

```
SC_CTOR(plic, plic_cfg cfg = plic_cfg())
: num_sources_p("num_sources", cfg.num_sources, "...")
, num_contexts_p("num_contexts", cfg.num_contexts, "...")
, access_delay_p("access_delay_ns", 2.0, "...")
{ }
```

Existing testbenches continue to work unchanged. New testbenches can use the CCI preset mechanism.

Phase 2 — Publish Parameters for Introspection

Register a global broker in `plic_tb.cpp` so that tool integrations work. No PLIC model changes needed.

Phase 3 — Deprecate `plic_cfg`

Once all callers use CCI presets, remove `plic_cfg` from the public API. The PLIC constructor becomes:

```
explicit plic(sc_core::sc_module_name name);
```

All configuration flows through the broker.

Phase 4 — Add Metadata and Callbacks

Enrich parameters with `add_metadata` calls and validation callbacks. This is purely additive.

8. Build Integration

The CCI library is already installed at `/Users/pdroy/cci`. Integration into the existing CMake build requires minimal changes:

```
# In CMakeLists.txt (top-level)

# Point CMake to the CCI installation
list(APPEND CMAKE_PREFIX_PATH "/Users/pdroy/cci")

# Find the package
find_package(SystemCCCI REQUIRED)

# Link the PLIC library against CCI
target_link_libraries(smc_plic PUBLIC SystemC::CCI)

# Include directories are transitively provided by the imported target
```

The installed CCI package provides a proper CMake config (`/Users/pdroy/cci/lib/cmake/SystemCCCI/SystemCCCIConfig.cmake`) that exposes the `SystemC::CCI` imported target automatically.

9. Summary

Topic	Current PLIC	With CCI
Configuration mechanism	<code>plic_cfg</code> struct passed at construction	<code>cci_param<T></code> members auto-registered with broker
Tool visibility	None — opaque C++ struct	Full: enumerate, read, write, describe via broker API
Preset injection	Caller must build custom struct	<code>broker.set_preset_cci_value()</code> before DUT construction
Run-time introspection	Not possible	<code>broker.get_param_handles()</code> → full name/value/description
Change audit trail	None	<code>h.get_value_origin().name()</code> → who set it
Validation	Constructor-level <code>SC_REPORT_FATAL</code> only	Pre-write callbacks; centralised, reusable validators
Documentation	Header comments only	First-class descriptions in the parameter object
Metadata	None	Arbitrary key-value map attached to each parameter
Build dependency	None	Link against <code>SystemC::CCI</code> (installed at <code>/Users/pdroy/cci</code>)
Backward compatibility	—	Phase 1 migration keeps <code>plic_cfg</code> intact

CCI turns the PLIC model from a self-contained C++ object with hard-coded configuration into a **standard-compliant, introspectable, tool-aware IP block** that can be configured, monitored, and validated consistently across any platform that supports SystemC.